

MANUAL TÉCNICO DEL SISTEMA LOSTIFY

Sistema de Gestión de Objetos Perdidos y Encontrados

1. INTRODUCCIÓN

Lostify es un sistema web desarrollado para facilitar el registro, búsqueda y administración de objetos perdidos y encontrados dentro de una institución educativa.

El sistema permite que los usuarios registren reportes de objetos extraviados o encontrados, almacenen evidencia fotográfica, consulten reportes existentes y gestionen su información personal.

La solución fue desarrollada utilizando Laravel como framework principal del backend, MySQL como sistema gestor de base de datos y tecnologías web para la interfaz de usuario.

2. OBJETIVO DEL SISTEMA

Objetivo General

Desarrollar una plataforma informática que permita gestionar eficientemente los objetos perdidos y encontrados mediante reportes digitales.

Objetivos Específicos

- Registrar objetos perdidos o encontrados.
- Almacenar información estructurada sobre cada reporte.
- Facilitar la búsqueda de objetos mediante filtros.
- Gestionar usuarios registrados.
- Proporcionar una API REST para futuras integraciones.
- Garantizar la seguridad y consistencia de los datos.

3. DESCRIPCIÓN GENERAL DEL SISTEMA

Lostify es una aplicación basada en arquitectura cliente-servidor.

El sistema está compuesto por:

Frontend

Encargado de la interacción con el usuario.

Tecnologías:

- HTML
- CSS

- JavaScript

Backend

Encargado de procesar la lógica de negocio.

Tecnologías:

- PHP
- Laravel

Base de Datos

Encargada de almacenar toda la información.

Tecnología:

- MySQL

4. ARQUITECTURA DEL SISTEMA

El sistema sigue una arquitectura de tres capas.

- PRESENTACIÓN
(HTML, CSS, JS)
- LÓGICA DE NEGOCIO
(Laravel)
|
- BASE DE DATOS
(MySQL)

Capa de Presentación

Permite la interacción entre el usuario y el sistema.

Funciones:

- Mostrar formularios.
- Mostrar reportes.
- Mostrar mensajes.
- Validaciones básicas.

Capa de Negocio

Implementada mediante Laravel.

Funciones:

- Procesar peticiones.
- Ejecutar validaciones.
- Gestionar autenticación.
- Administrar reportes.
- Administrar usuarios.

Capa de Datos

Implementada en MySQL.

Funciones:

- Almacenar usuarios.
- Almacenar objetos.
- Almacenar reportes.
- Almacenar categorías.
- Almacenar estados.
- Almacenar ubicaciones.

5. TECNOLOGÍAS UTILIZADAS

Laravel

Framework PHP utilizado para desarrollar la lógica del sistema.

Características utilizadas:

- Migraciones
- Seeders
- Factories
- Middleware
- Validaciones
- Eloquent ORM
- API REST

PHP

Lenguaje principal del backend.

Utilizado para:

- Controladores
- Modelos
- Validaciones
- Manejo de excepciones

MySQL

Sistema de gestión de bases de datos.

Utilizado para:

- Almacenamiento persistente.
- Relaciones entre tablas.

HTML

Utilizado para:

- Formularios.
- Estructura de páginas.
- Componentes visuales.

CSS

Utilizado para:

- Diseño responsivo.
- Estilos visuales.
- Animaciones.

JavaScript

Utilizado para:

- Consumo de API REST.
- Filtros dinámicos.

Laravel Sanctum

Utilizado para:

- Autenticación API.
- Generación de Tokens.
- Protección de endpoints.

6. PATRÓN DE DISEÑO IMPLEMENTADO

Modelo

Representa la estructura de datos.

Modelos utilizados:

- User
- Objeto
- Reporte
- Categoria
- Estado

- Ubicacion
- Rol

Controlador

Gestiona la lógica de negocio.

Controladores:

- AuthController
- ReporteController
- ObjetoController
- CategoriaController
- EstadoController
- PerfilController

7. ESTRUCTURA DE BASE DE DATOS

La base de datos se denomina: Lostify

Tabla Roles

Almacena los tipos de usuario.

Campos:

- id
- nombre

Ejemplos:

- Administrador
- Usuario

Tabla Users

Almacena la información de los usuarios.

Campos principales:

- id
- name
- apellido_paterno
- apellido_materno
- email
- password
- rol_id

Tabla Categorías

Almacena categorías de objetos.

Ejemplos:

- Electrónicos
- Mochilas
- Documentos
- Accesorios

Tabla Estados

Almacena el estado del objeto.

Ejemplos:

- Perdido
- Encontrado
- Entregado

Tabla Ubicaciones

Almacena ubicaciones válidas.

Ejemplos:

- Biblioteca
- Cafetería
- Aula A101

Su implementación evita errores de escritura y garantiza consistencia en los registros.

Tabla Objetos

Almacena información del objeto.

Campos:

- id
- nombre_obj
- descripcion
- fechaenco
- imagen
- categoria_id
- estado_id
- ubicacion_id

Tabla Reportes

Almacena reportes realizados por usuarios.

Campos:

- id
- descripcion
- fecha_reporte
- user_id
- objeto_id

8. RELACIONES DE BASE DE DATOS

Usuario - Reportes

Relación:

1 a N

Un usuario puede tener muchos reportes.

Reporte - Objeto

Relación:

1 a 1

Cada reporte pertenece a un objeto.

Categoría - Objetos

Relación:

1 a N

Estado - Objetos

Relación:

1 a N

Ubicación - Objetos

Relación:

1 a N

9. API REST IMPLEMENTADA

El sistema cuenta con una API REST para la gestión de recursos.

Ruta base: <http://127.0.0.1:8000/api>

AUTENTICACIÓN

Registro

POST /api/register

Permite crear un nuevo usuario.

Login

POST /api/login

Permite autenticar un usuario.

OBJETOS

Listar Objetos

GET /api/objetos

Crear Objeto

POST /api/objetos

Mostrar Objeto

GET /api/objetos/{id}

Actualizar Objeto

PUT /api/objetos/{id}

Eliminar Objeto

DELETE /api/objetos/{id}

REPORTES

Obtener Reportes

GET /api/reportes

Crear Reporte

POST /api/reportes

Actualizar Reporte

PUT /api/reportes/{id}

Eliminar Reporte

DELETE /api/reportes/{id}

10. VALIDACIONES IMPLEMENTADAS

Las validaciones se realizan mediante Laravel.

Ejemplo:

```
$request->validate([
    'nombre' => 'required|max:100',
    'descripcion' => 'required|max:255',
    'categoria_id' => 'required|exists:categorias,id',
    'estado_id' => 'required|exists:estados,id',
    'ubicacion_id' => 'required|exists:ubicaciones,id'
]);
```

Beneficios:

- Evita datos incorrectos.
- Mantiene integridad de la base de datos.
- Mejora la experiencia del usuario.

11. MANEJO DE EXCEPCIONES

Para evitar errores inesperados se utiliza:

```
try{
}
catch(Exception $e){
}
```

Funciones:

- Capturar errores.
- Mostrar mensajes amigables.
- Evitar caídas del sistema.

12. SEGURIDAD

El sistema implementa varias medidas de seguridad.

Hash de Contraseñas

Laravel utiliza:

Hash::make()

para almacenar contraseñas cifradas.

Protección CSRF

Protección contra ataques Cross Site Request Forgery.

@csrf

Sanctum

Permite proteger endpoints API.

14. SEEDERS Y FACTORIES

Se utilizaron para generar datos iniciales.

Seeders implementados:

- RolSeeder
- EstadoSeeder
- CategoriaSeeder
- UbicacionSeeder

Factories implementados:

- UserFactory
- ObjetoFactory
- ReporteFactory

Objetivo:

- Facilitar pruebas.
- Generar registros automáticos.

15. PRUEBAS DEL SISTEMA

Se utilizó Thunder Client para verificar:

Registro

POST /api/register

Login

POST /api/login

Objetos

GET /api/objetos

Reportes

GET /api/reportes

16. INSTALACIÓN DEL SISTEMA

Instalar Dependencias

- composer install

Configurar Variables

Archivo: .env

Ejecutar Migraciones

- php artisan migrate

Ejecutar Seeders

- php artisan db:seed

Iniciar Servidor

- php artisan serve

18. CONCLUSIONES

Se desarrolló exitosamente el sistema Lostify, cumpliendo con el objetivo general de gestionar de manera eficiente los objetos perdidos y encontrados mediante reportes digitales. La implementación del sistema permitió centralizar la información en una única plataforma, facilitando el registro, consulta y administración de los reportes realizados por los usuarios.

Respecto a los objetivos específicos planteados, se logró implementar un módulo para el registro de objetos perdidos o encontrados, permitiendo almacenar información detallada y estructurada de cada reporte, incluyendo descripción, categoría, estado, ubicación e imagen del objeto. Asimismo, se desarrolló un sistema de búsqueda mediante filtros que mejora significativamente la localización de objetos registrados dentro de la plataforma.

De igual manera, se implementó un módulo de gestión de usuarios que permite el registro, autenticación y administración de perfiles, garantizando que únicamente usuarios autorizados puedan acceder a las funcionalidades del sistema. Además, se desarrolló una

API REST que facilita futuras integraciones con aplicaciones móviles u otros sistemas externos, incrementando la escalabilidad del proyecto.

Finalmente, mediante el uso de validaciones, relaciones de base de datos, manejo de excepciones y mecanismos de autenticación, se logró garantizar la seguridad, integridad y consistencia de la información almacenada. En consecuencia, Lostify constituye una solución tecnológica funcional, escalable y mantenible que contribuye a optimizar el proceso de recuperación y gestión de objetos perdidos y encontrados dentro de una institución.